# Métodos orientados a objetos

## Analise e Modelagem de Sistemas

Eduardo Furlan Miranda

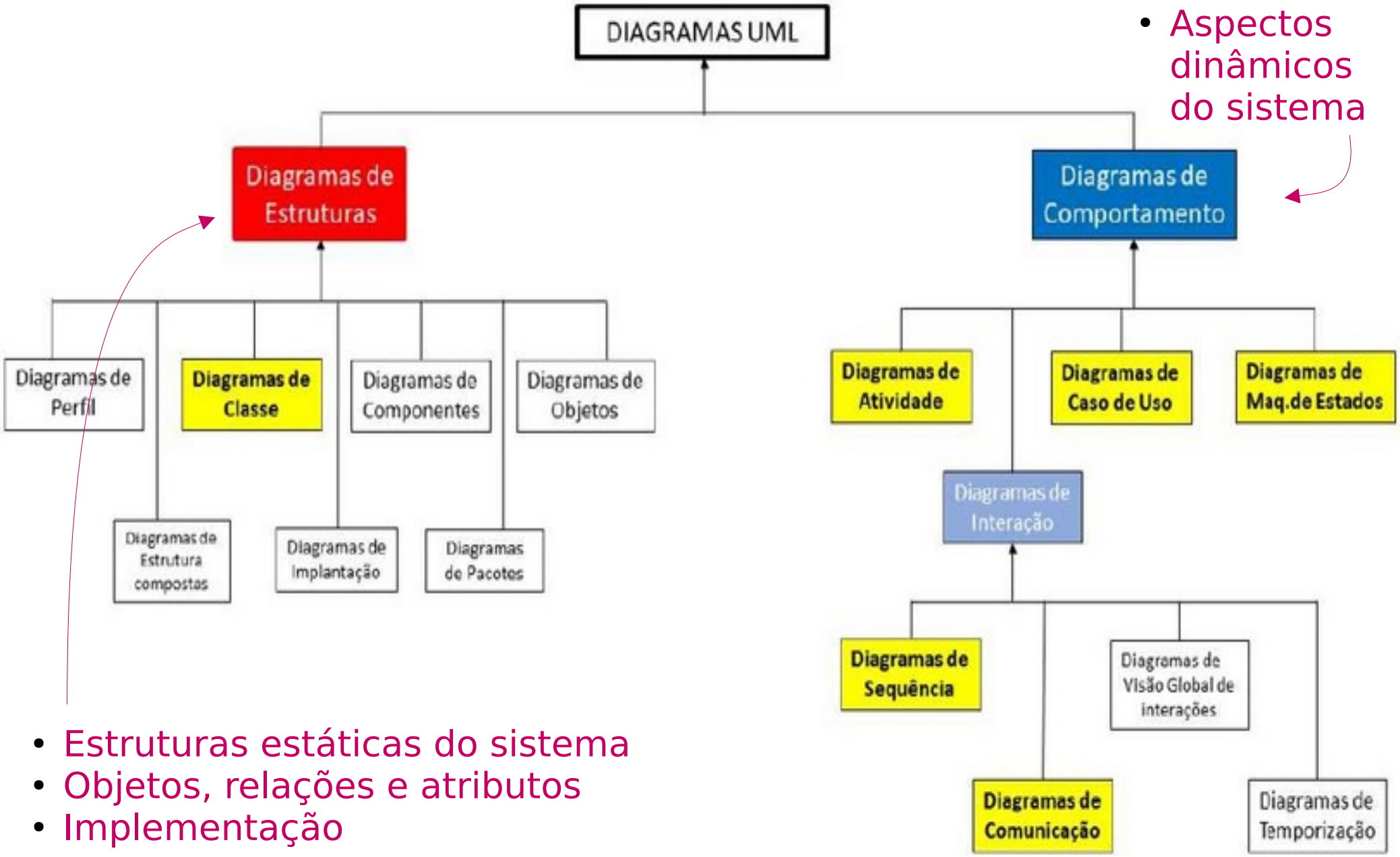
2025-11-03

Adaptado de: WERLICH, C. Análise e Modelagem de Sistemas.
Londrina: EDE SA, 2020. ISBN 978-85-522-1683-4.

# UML (Unified Modeling Language)

- Linguagem padrão de modelagem visual usada em engenharia de software para especificar, visualizar, construir e documentar sistemas de software
- Basicamente a UML é composta de diagramas
- Usa o estilo orientado a objetos (OO)
- As especificações UML são controladas pela OMG (Object Management Group)
- 14 diagramas subdivididos em duas categorias
  - Diagramas de estrutura
  - Diagramas de comportamento

# Diagramas UML



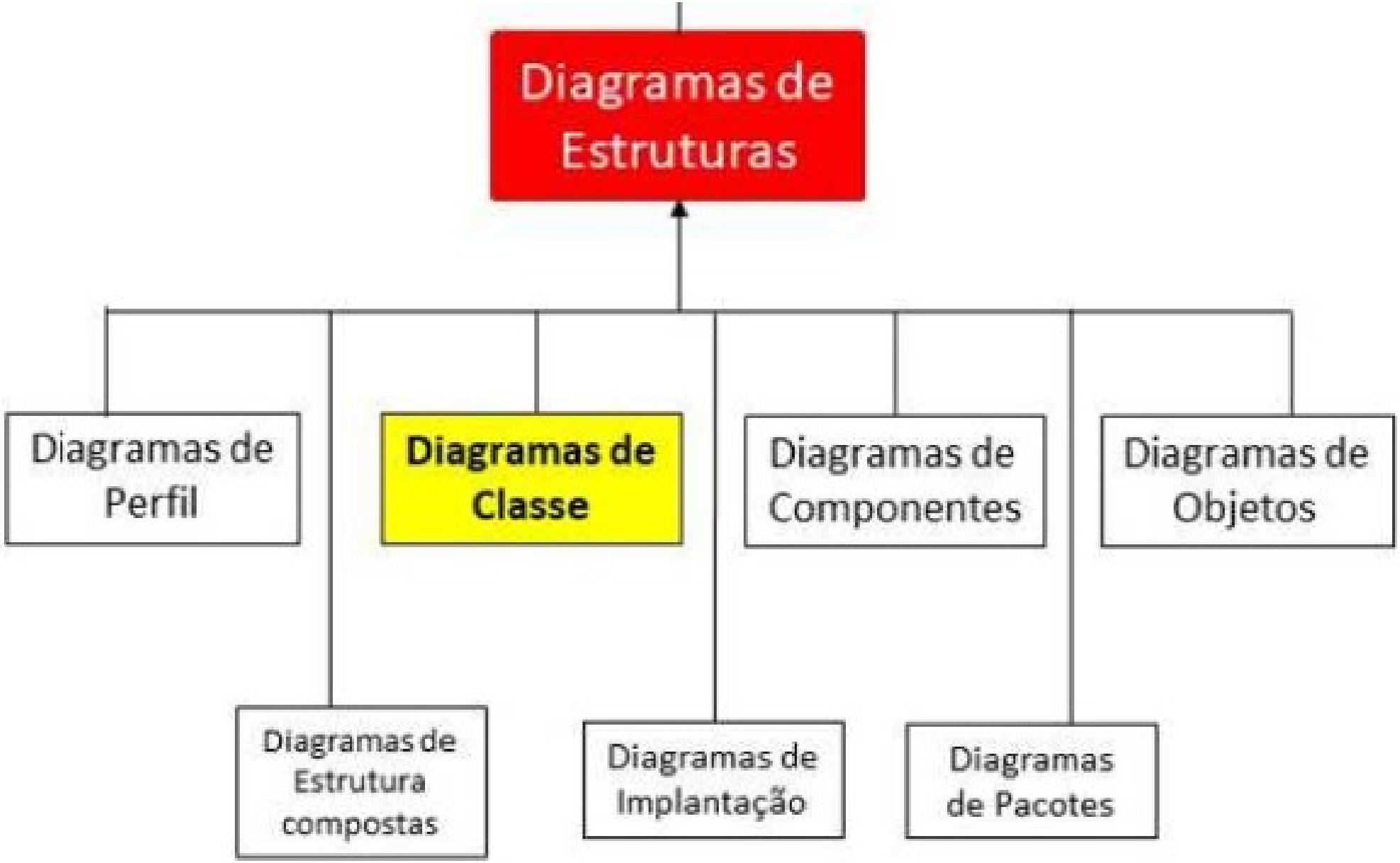

- Aspectos dinâmicos do sistema

- Estruturas estáticas do sistema
- Objetos, relações e atributos
- Implementação

# Detalhe do diagrama anterior



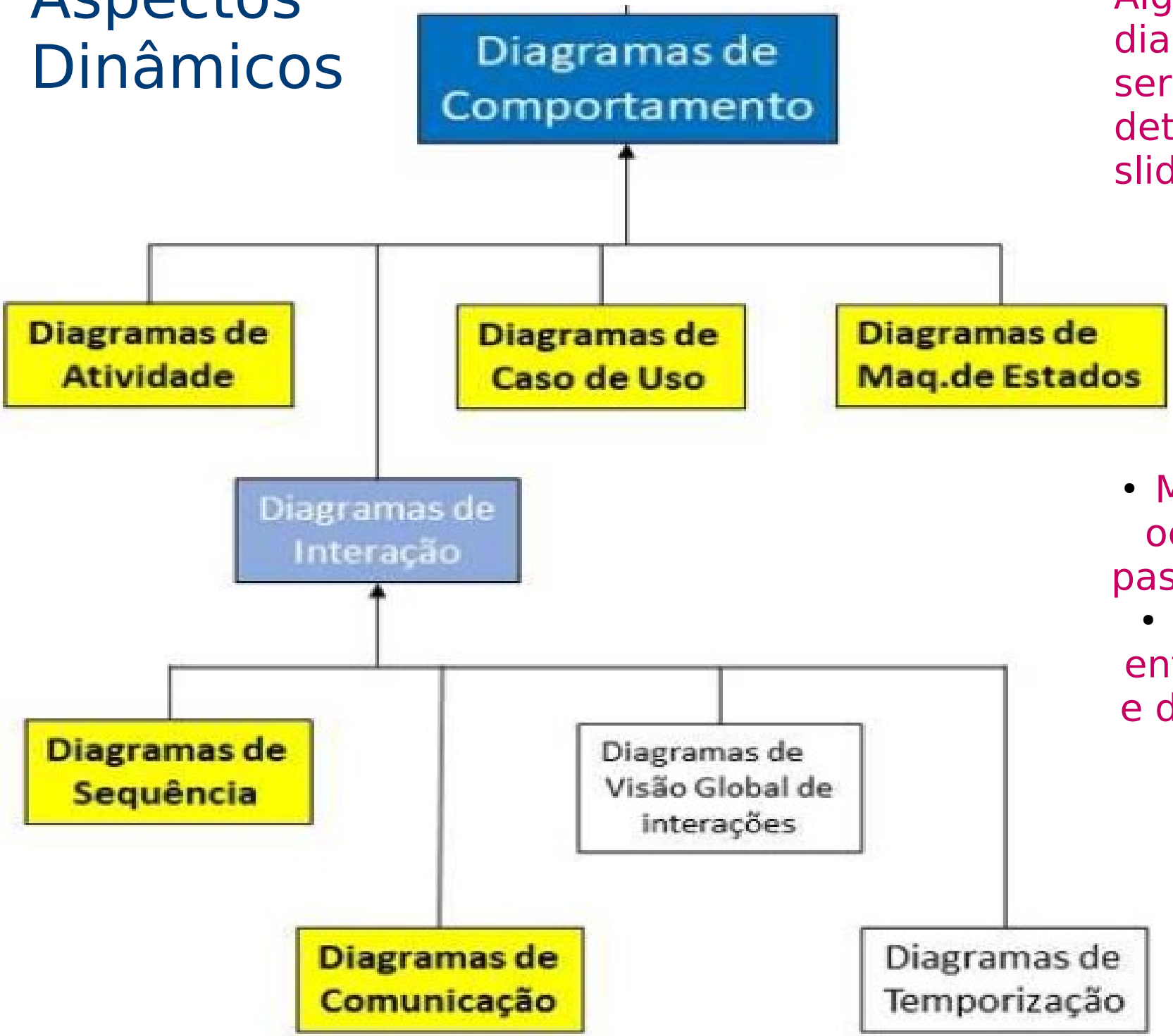
## Estruturas estáticas do sistema



# Aspectos Dinâmicos

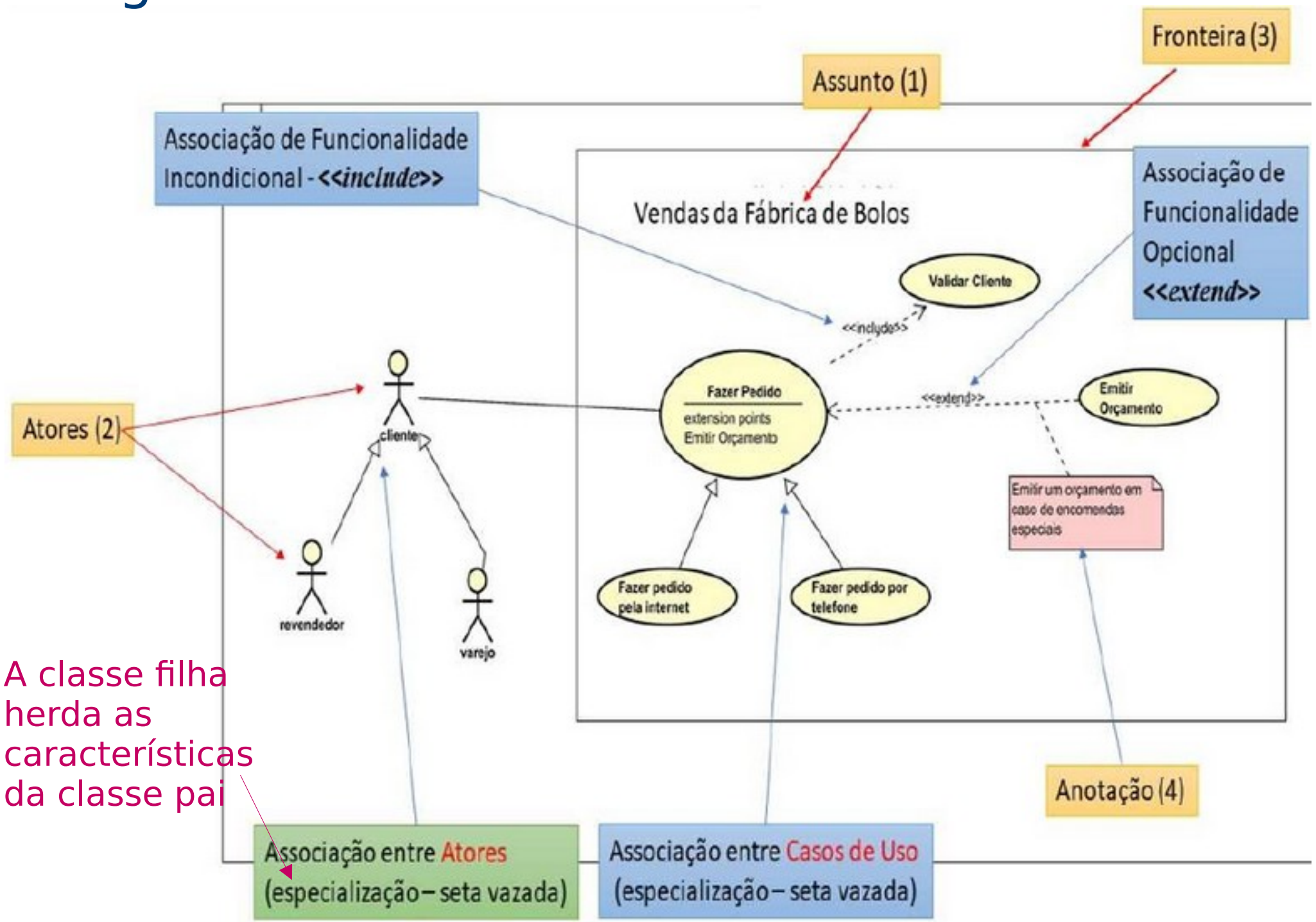


Alguns diagramas serão detalhados nos slides a seguir




- Mudanças que ocorrem com o passar do tempo
- Comunicação entre os objetos e das mudanças de estados internos e/ou externos no sistema

# Diagrama de caso de uso




A classe filha herda as características da classe pai

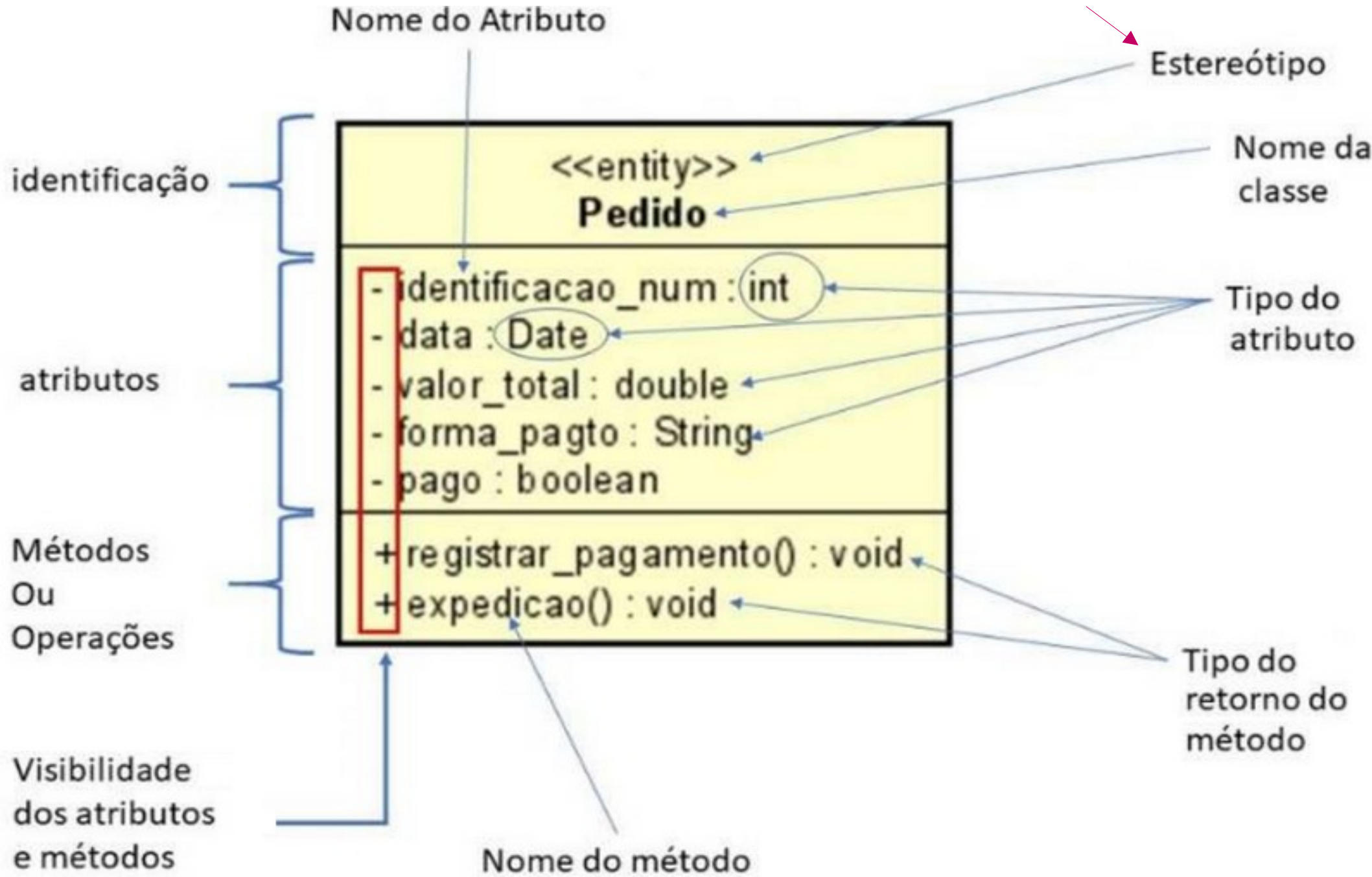
# Diagrama de caso de uso

- Fornece uma visão geral dos objetivos que os usuários (os atores) desejam alcançar utilizando o sistema

- Os elementos mais importantes são
  - Atores
  - Relacionamentos
  - Fluxo de eventos

# Diagrama de classe

Permite classificar elementos do diagrama que tenham algo em comum



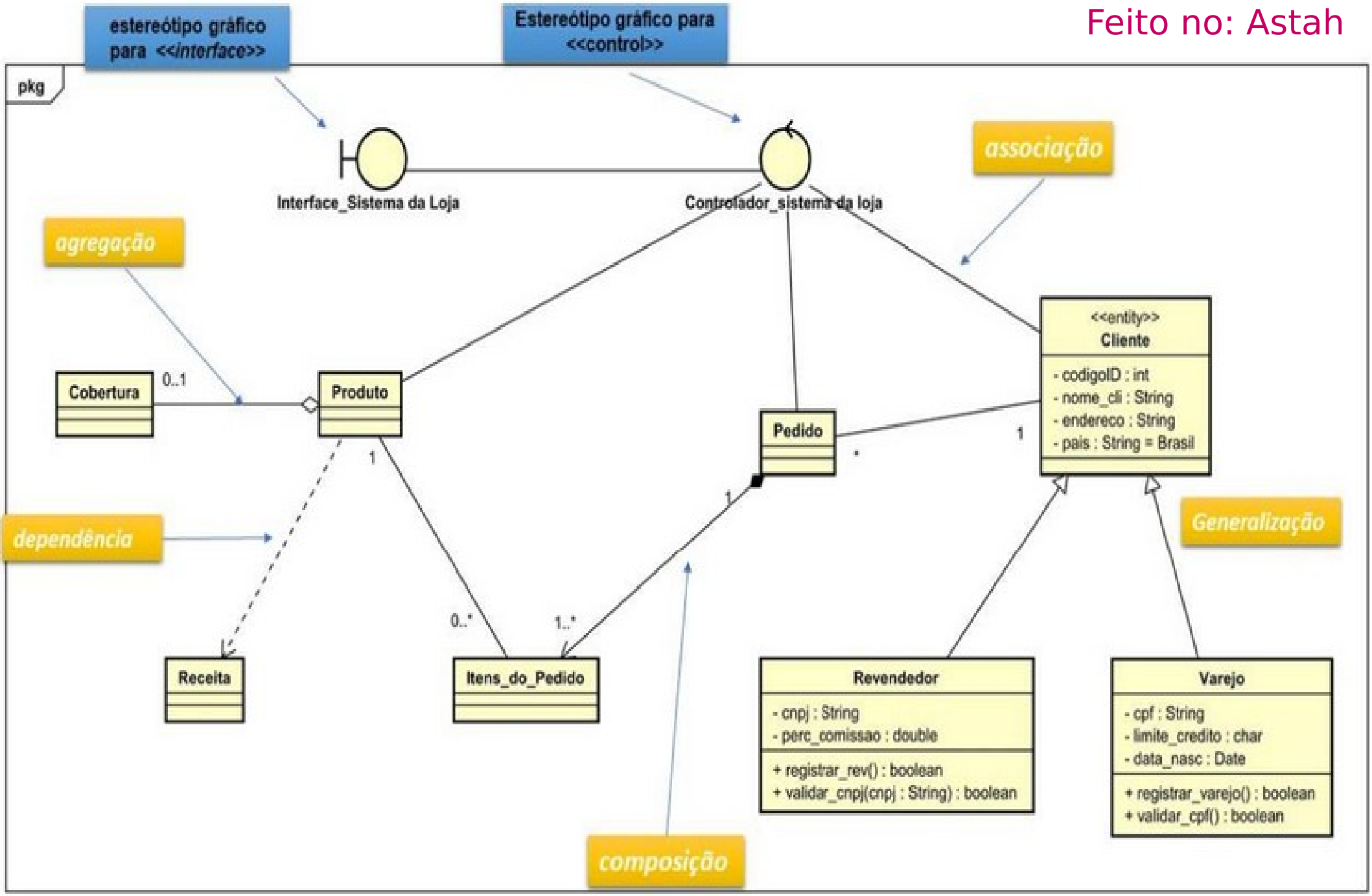
- Um mecanismo de extensibilidade da UML bastante utilizado nos diagramas de classe são os estereótipos
  - Permite classificar elementos do diagrama que tenham algo em comum
  - Podem ser
    - Definidos pelo desenvolvedor
    - Predefinidos (nativos na linguagem)
      - A UML dispõe de 50 estereótipos predefinidos
        - 3 são mais comuns
          - <<entity>> armazena dados
          - <<boundary>> comunicação
          - <<control>> regras

# Relacionamento entre classes

- Dependência — se mudar uma, afeta a outra
- Associação — relacionamento genérico
- Agregação — o filho independe do pai
- Composição — se apaga o pai o filho fica sem sentido
- Generalização/especialização — o filho herda do pai
- Multiplicidade — associação de várias instâncias

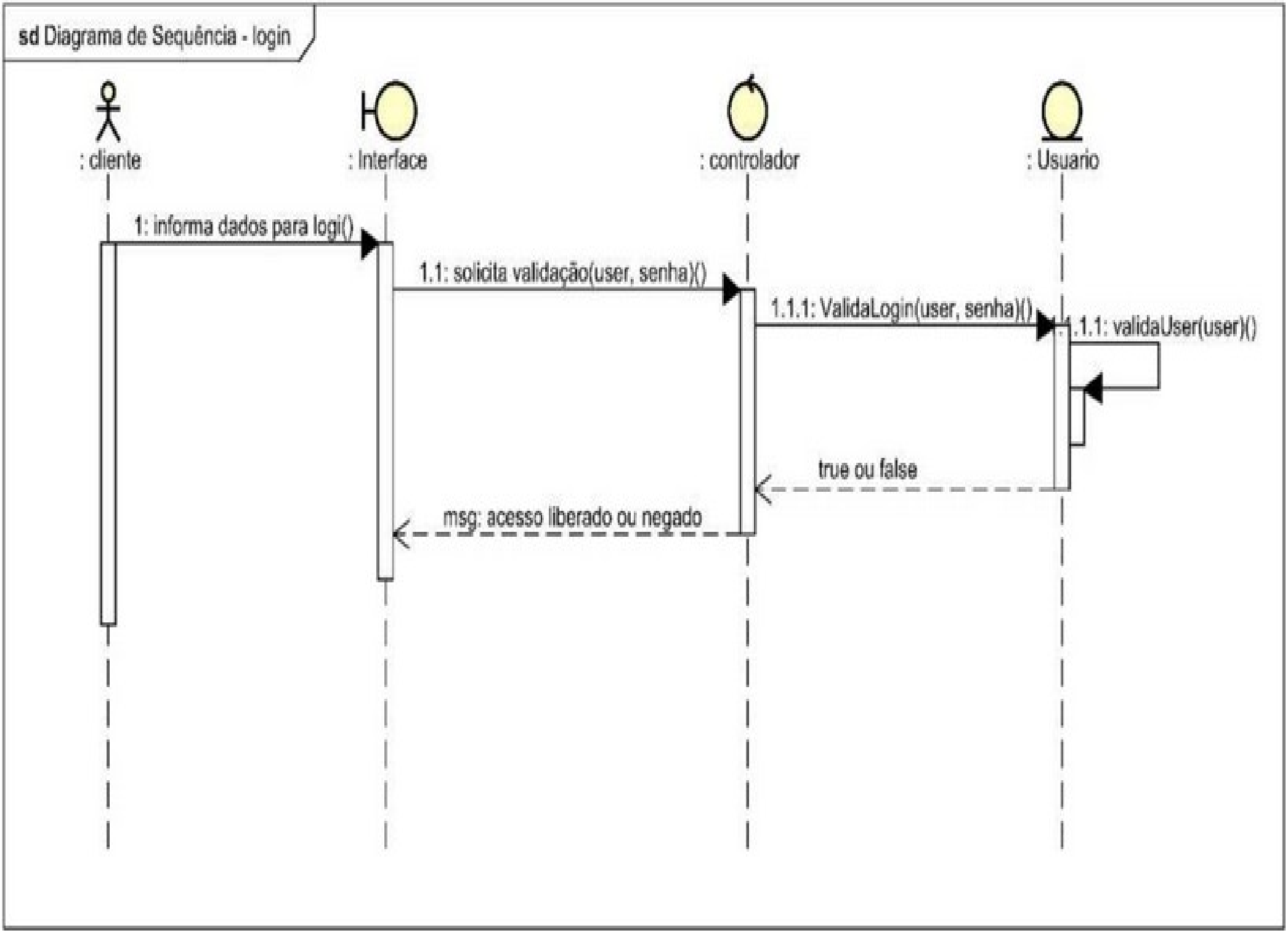
# Diagrama de classe

Feito no: Astah



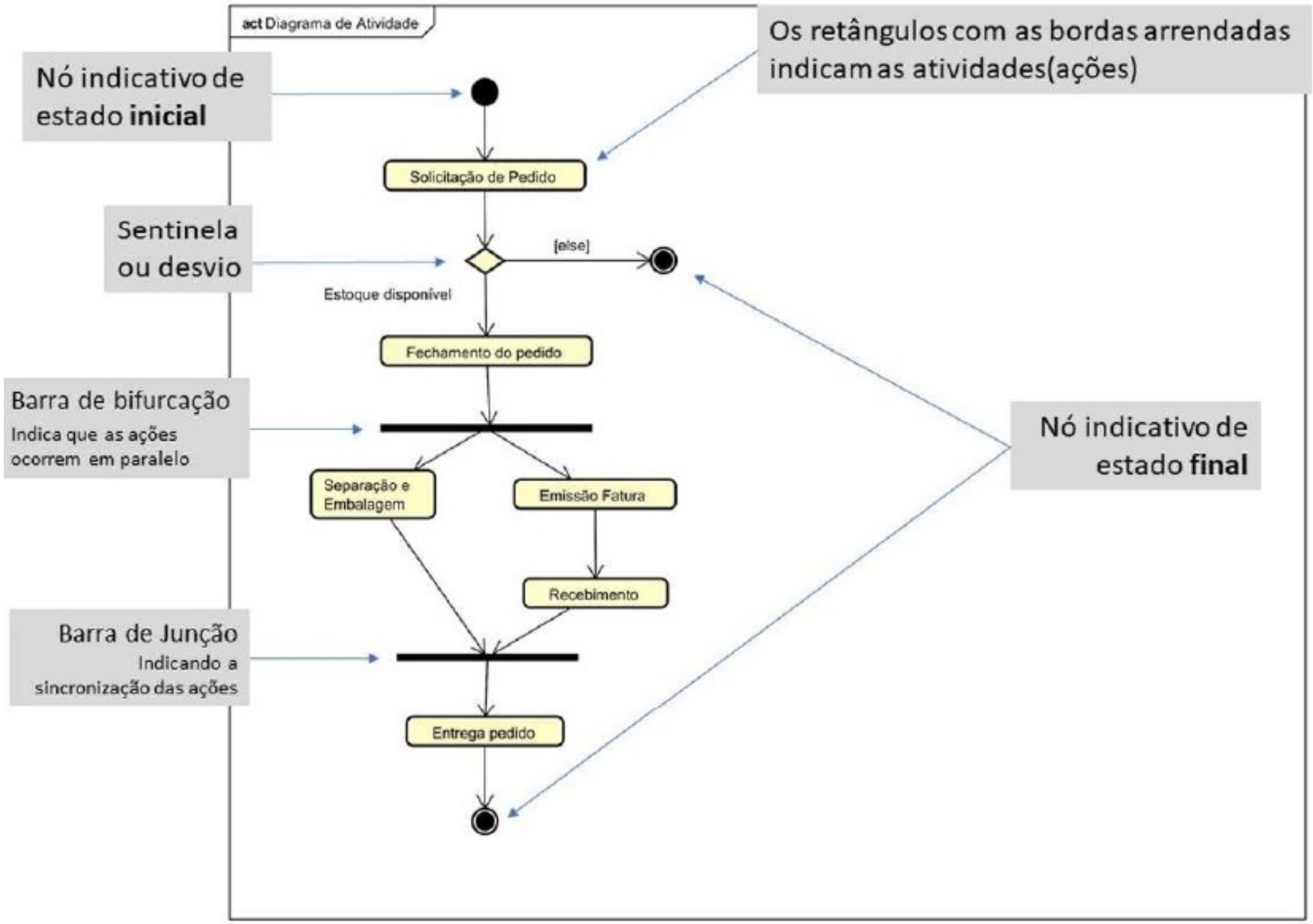
# Diagrama de sequência

- Entre os diagramas de interação, esse é o mais utilizado
- Mostra a interação entre os participantes do cenário ao longo da vida, a sequência de passos dos objetos
  - Verticalmente e na ordem de cima para baixo
- (Vide próximo slide)
  - Os objetos participantes da interação são organizados na horizontal
  - Linha da vida: cada linha tem seu foco de controle
  - O ator cliente acessa o sistema e fornece os dados para efetuar login na interface
  - A interface aciona o controlador que, por sua vez, aciona uma instância da classe usuário para validar o acesso
  - O objeto usuário retorna se a validação foi aceita ao controlador, que por sua fez envia à interface, e o acesso é liberado ao cliente


sd Diagrama de Sequência - login
: cliente
: Interface
: controlador
: Usuario
1: informa dados para logi()
1.1: solicita validação(user, senha)()
1.1.1: ValidaLogin(user, senha)()
1.1.1: validaUser(user)()
true ou false
msg: acesso liberado ou negado

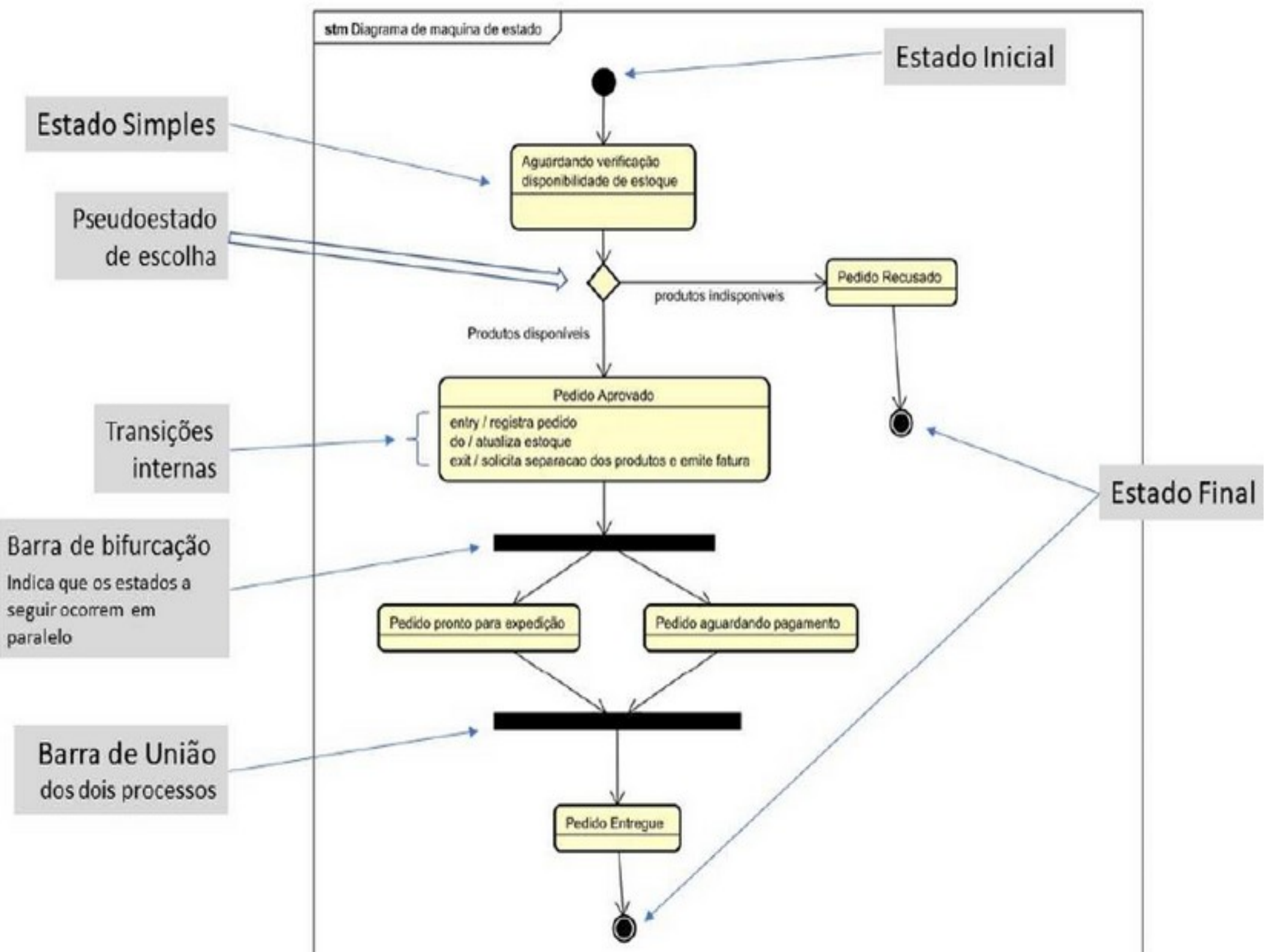
# Diagrama de atividade

- Descreve os passos que devem ser seguidos para a execução de uma determinada atividade
- Um fluxograma com atividades em paralelo
- Elementos básicos: ações (atividades), sentinela (desvios), estados inicial e final, barra de bifurcação e barra de junção


act Diagrama de Atividade
Nó indicativo de estado **inicial**
Os retângulos com as bordas arrendadas indicam as atividades(ações)
Solicitação de Pedido
Sentinela ou desvio
[else]
Estoque disponível
Fechamento do pedido
Barra de bifurcação
Indica que as ações ocorrem em paralelo
Separação e Embalagem
Emissão Fatura
Recebimento
Nó indicativo de estado **final**
Barra de Junção
Indicando a sincronização das ações
Entrega pedido

# Diagrama de máquina de estados

- Mostra a transição de um estado a outro dos objetos do sistema
- Símbolos de estado inicial, estado final, barra de bifurcação e pseudoestado (desvio), estado que apresenta transições internas
- Transições internas de estado:
  - Entry: indica ações internas do estado executada pelo objeto ao assumir um estado
  - Do: são ações internas do estado executadas durante o período que o objeto se encontra em um estado
  - Exit: neste caso, são ações executadas pelo objeto quando ele sai de
  - um estado


stm Diagrama de maquina de estado
Estado Inicial
Estado Simples
Aguardando verificação disponibilidade de estoque
Pseudoestado de escolha
Pedido Recusado
produtos indisponiveis
Produtos disponiveis
Pedido Aprovado
entry / registra pedido
do / atualiza estoque
exit / solicita separacao dos produtos e emite fatura
Transições internas
Estado Final
Barra de bifurcação
Indica que os estados a seguir ocorrem em paralelo
Pedido pronto para expedição
Pedido aguardando pagamento
Barra de União
dos dois processos
Pedido Entregue

# Desenvolvimento ágil

- Manifesto Ágil
  - Declaração de valores e princípios essenciais para o desenvolvimento de software
- Focos (alguns)
  - Responder rapidamente à entrega do sistema
  - Reuso e principalmente utilização de componentes
  - Desenvolvimento de sistemas críticos
- Métodos comuns
  - eXtreme Programming
  - Scrum

# Princípios dos métodos ágeis

| PRINCÍPIOS | DESCRIÇÃO |
|---|---|
| **Envolvimento do cliente** | Clientes devem ser profundamente envolvidos no processo de desenvolvimento. Seu papel é fornecer e priorizar novos requisitos do sistema e avaliar as iterações do sistema. |
| **Entrega incremental** | O software é desenvolvido em incrementos e o cliente especifica os requisitos a serem incluídos em cada incremento. |
| **Pessoas, não processo** | As habilidades da equipe de desenvolvimento devem ser reconhecidas e exploradas. Os membros da equipe devem desenvolver suas próprias maneiras de trabalhar sem processos prescritivos. |
| **Aceite às mudanças** | Tenha em mente que os requisitos do sistema vão mudar, por isso projete o sistema para acomodar essas mudanças. |
| **Manutenção da simplicidade** | Concentre-se na simplicidade do software que está sendo desenvolvido e do processo de desenvolvimento. Sempre que possível, trabalhe ativamente para eliminar a complexidade do sistema. |

# Extreme Programming (XP)



## Método ágil de desenvolvimento de software

- O envolvimento do cliente é parte do processo
- Adaptável a mudanças
- Busca a simplicidade do software
- Foco no valor do negócio
- Colaborativo (equipe)
- Desenvolvimento do código é feito aos pares
  - Permite que um programador mais experiente atue junto
  - Duas pessoas pesando juntos

# Scrum

## Método ágil de gerenciamento de projetos

- Utiliza uma estrutura de reuniões, papéis e artefatos para ajudar equipes a se autoorganizarem e entregarem produtos de forma iterativa e incremental
- Nomes específicos para algumas disciplinas
  - Sprint (marcos das iterações do Processo Unificado (PU), visto na seção 4.2)
- O tempo de cada sprint é relativamente curto e diariamente a equipe avalia os resultados
- Ao final de um sprint deve ser entregue um produto ao cliente (minissistema)